

Data-Driven Computer Animation

Lecture 4.

Forward/Inverse Dynamics

Overview

- Motivation
- Background / Notation
- Articulate Dynamics Algorithms
 - Newton-Euler Algorithm
 - Recursive Newton-Euler
 - Articulated-Body Algorithm (Featherstone)
- Motion synthesis with inverse dynamics

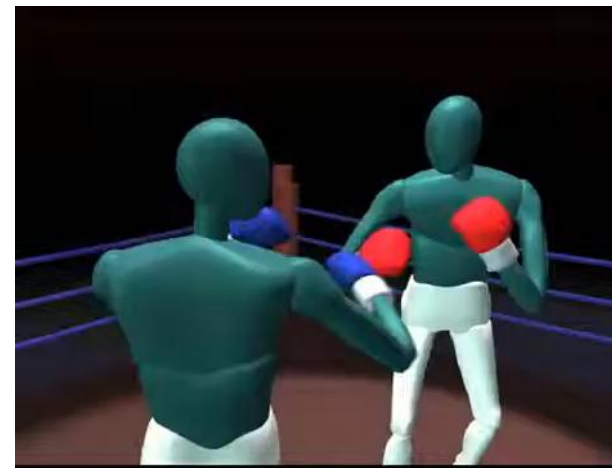
Character Animation

- There are three methods
 - Create them manually
 - Use real human / animal motions
 - Use physically based simulation



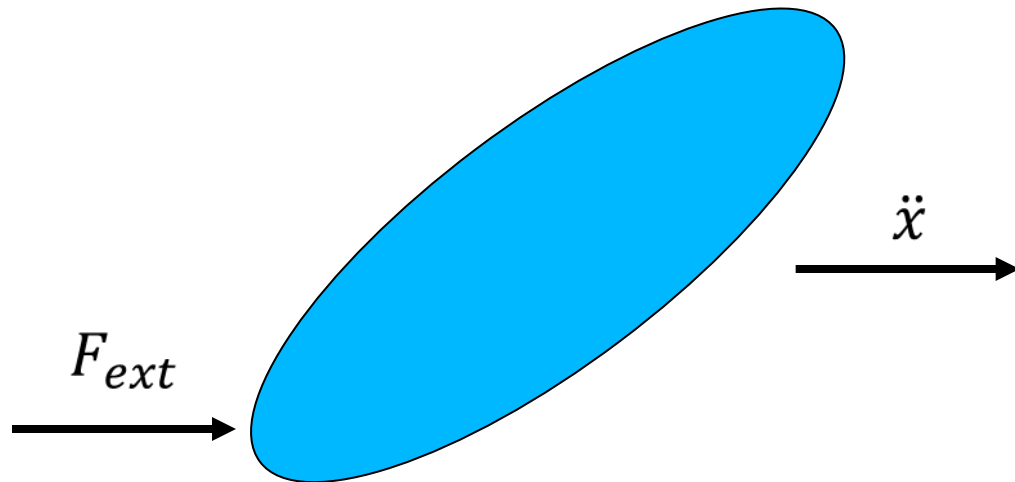
Physically-based Animation

- Forward/inverse dynamics
- Voluntary motion, passive motion
- PD control for voluntary motion, response motion



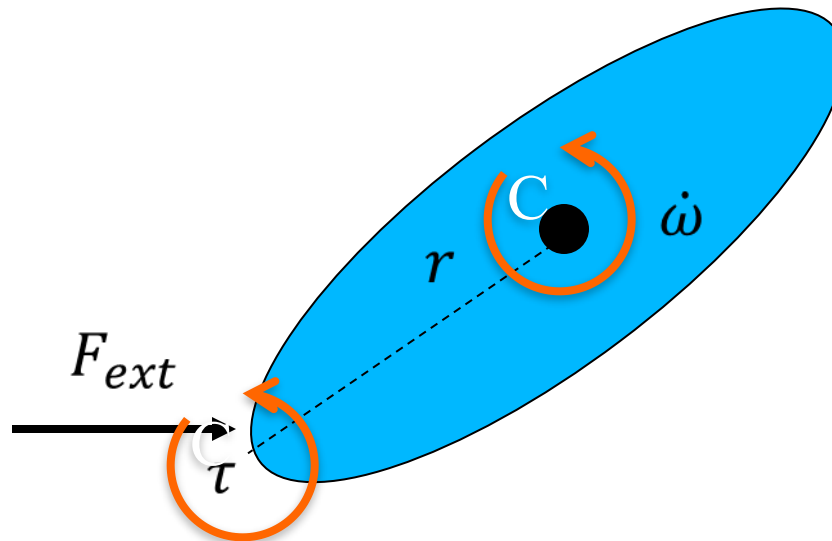
How to do the physics computation?

- For every rigid body, its motion follows Newton's law
- $F_{ext} = M\ddot{x}$ (linear motion)
- $r \times F_{ext} + \tau = I\dot{\omega} + \omega \times I\omega$ (angular motion)



How to do the physics computation?

- For every rigid body, its motion follows Newton's law
- $F_{ext} = M\ddot{x}$ (linear motion)
- $r \times F_{ext} + \tau = I\dot{\omega} + \omega \times I\omega$ (angular motion)

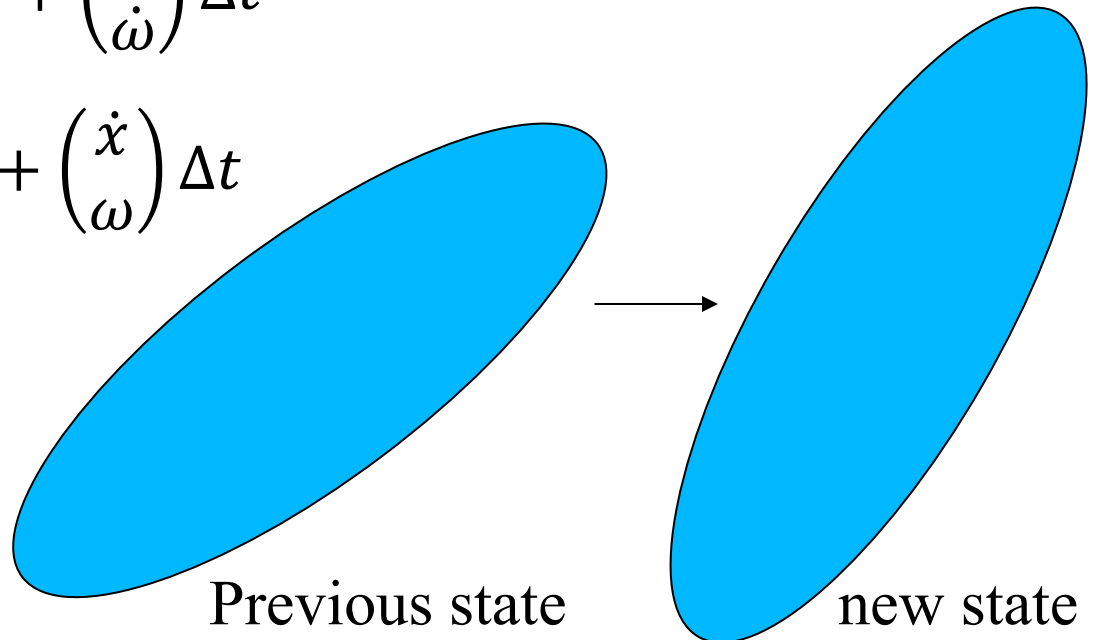


Physical Simulation

- Using the computed $\ddot{x}$, $\dot{\omega}$, update the position and orientation by integration

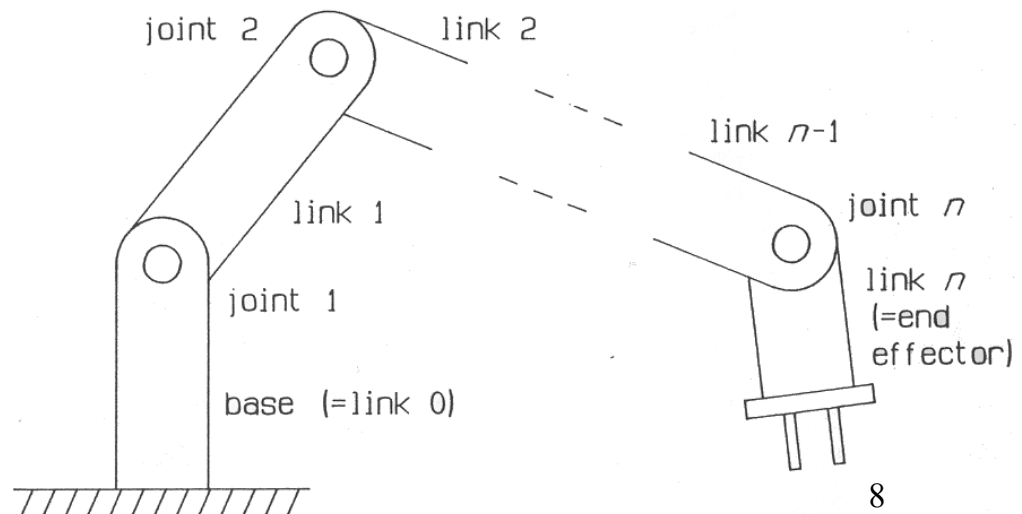
$$- \begin{pmatrix} \dot{x} \\ \omega \end{pmatrix} = \begin{pmatrix} \dot{x}_{prev} \\ \omega_{prev} \end{pmatrix} + \begin{pmatrix} \ddot{x} \\ \dot{\omega} \end{pmatrix} \Delta t$$

$$- \begin{pmatrix} x \\ \theta \end{pmatrix} = \begin{pmatrix} x_{prev} \\ \theta_{prev} \end{pmatrix} + \begin{pmatrix} \dot{x} \\ \omega \end{pmatrix} \Delta t$$



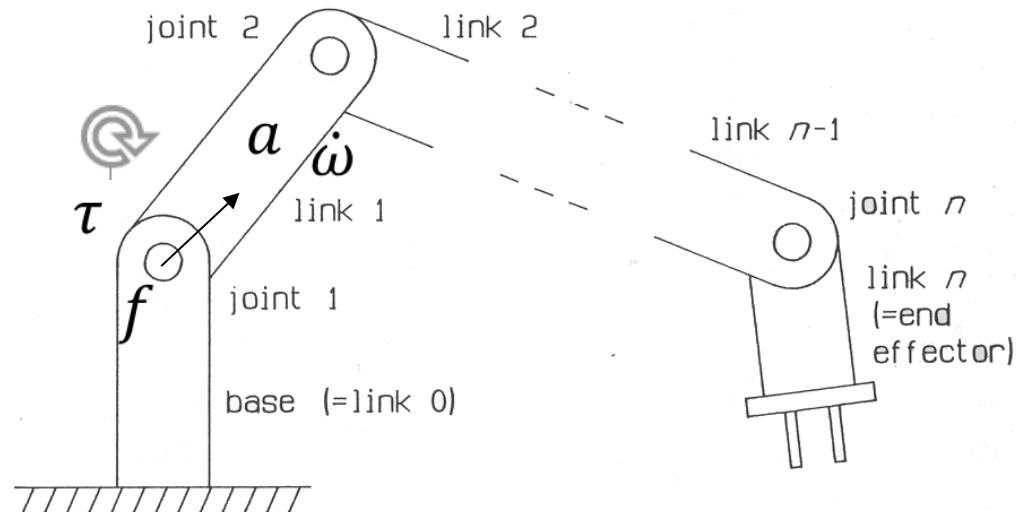
Articulated Body

- An articulated body is a group of rigid bodies (called links) connected by joints
- Multiple types of joints
 - Hinge/Revolute (1 degree of freedom)
 - Ball joint (3 degrees of freedom)
 - Prismatic, screw, etc.



Articulated Bodys

- For articulated bodies, there will be
 - Force between the joints
 - Torque generated by the motors at the joints
 - These forces and torques will result in a motion of the entire model

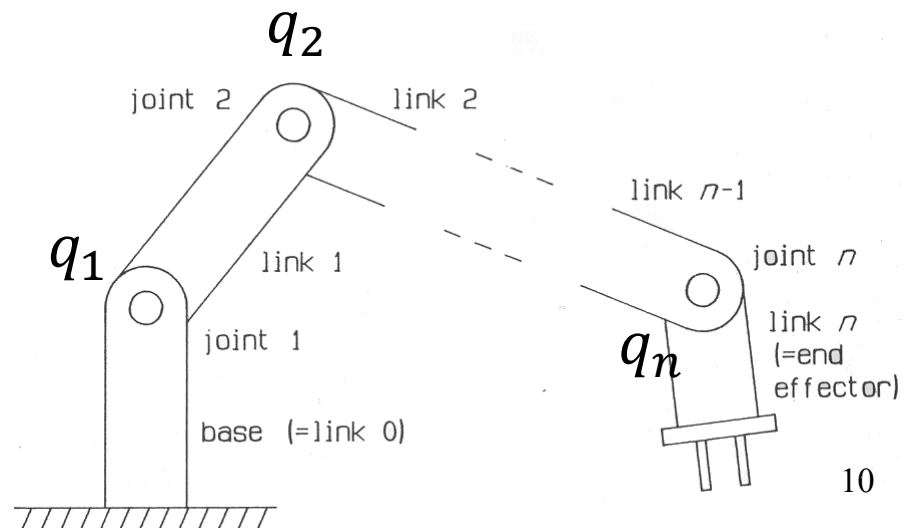


Notation : generalized coordinates

- Transitioning this to articulated bodies

$$\mathbf{q} = (q_1, q_2, \dots, q_n)$$

- Generalized coordinates
- All degrees of freedom put into a single vector

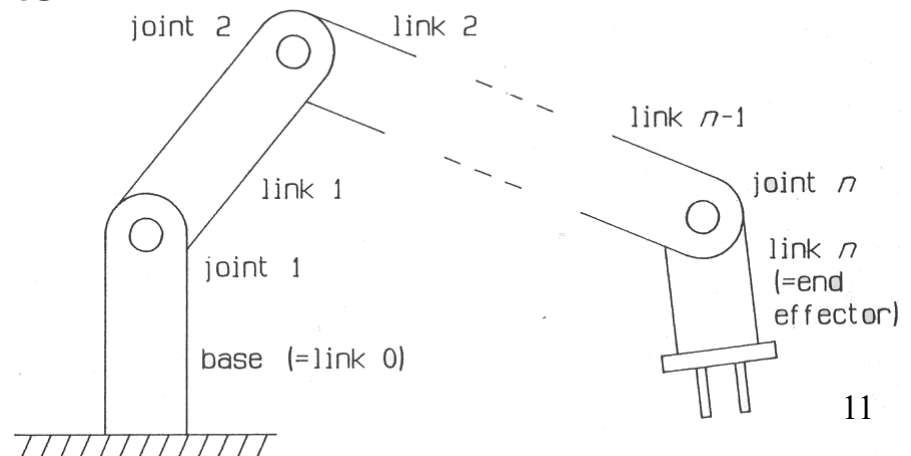


Notation

- Transitioning this to articulated bodies

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

- $\boldsymbol{\tau}$ is the force on link i
- $\mathbf{H}$ is the joint-space inertia matrix ($n \times n$)
- $\mathbf{q}$, $\dot{\mathbf{q}}$ and $\ddot{\mathbf{q}}$ are the coordinates, velocities and accelerations of the joints
- $\mathbf{C}$ are Coriolis force
- $\boldsymbol{\tau}_g$: force due to gravity



Forward vs. Inverse Dynamics

- Inverse Dynamics
 - The calculation of forces given a set of accelerations
- Forward Dynamics
 - The calculation of accelerations given a set of forces

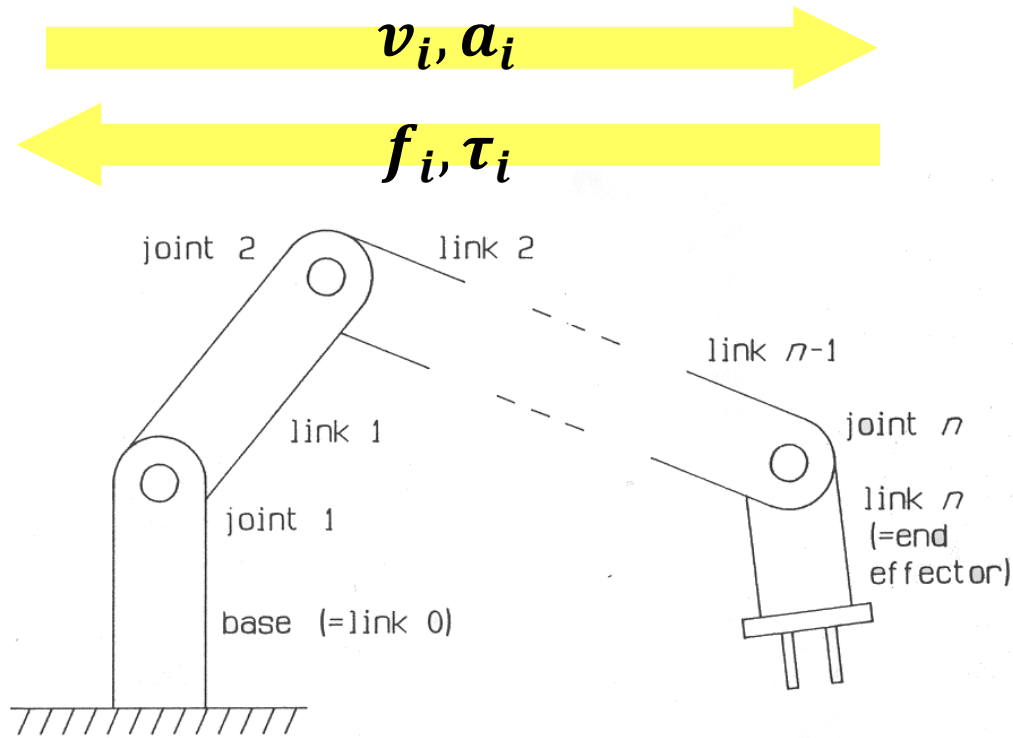

$$\tau = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \tau_g$$

Algorithms

- Inverse Dynamics
 - Newton-Euler Algorithm
- Forward Dynamics
 - Recursive Newton-Euler
 - Articulated-Body Algorithm

Inverse Dynamics: Newton-Euler Algorithm

- Goal
 - Given the accelerations and velocities at the joints, find the forces/torques required at the joints to generate those accelerations
- Recursive approach



- Compute v_i, a_i from root to leaf using $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$
- Compute the force f_i, τ_i from leaves to root using the equation of motion

Spatial Notation

- Spatial notation combines linear and angular quantities – extending the 3x3 notion to 6x6
- $\mathbf{v} = \begin{bmatrix} \boldsymbol{\omega} \\ \dot{\mathbf{x}} \end{bmatrix}$ $\boldsymbol{\omega}$ angular velocity, $\dot{\mathbf{x}}$ linear velocity
- $\mathbf{a} = \begin{bmatrix} \dot{\boldsymbol{\omega}} \\ \ddot{\mathbf{x}} \end{bmatrix}$ $\dot{\boldsymbol{\omega}}$ angular acceleration, $\ddot{\mathbf{x}}$ linear acceleration
- $\mathbf{f} = \begin{bmatrix} \mathbf{n} \\ \mathbf{f} \end{bmatrix}$ $\mathbf{n}$ momentum, $\mathbf{f}$ force
- $\mathbf{I} = \begin{bmatrix} \mathbf{I} & 0 \\ 0 & m\mathbf{1} \end{bmatrix}$ $\mathbf{I}$ moment of inertia around center of mass, m is mass

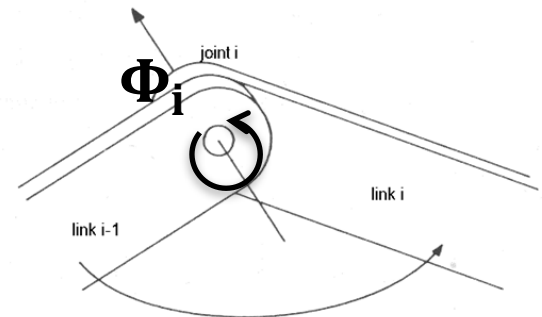
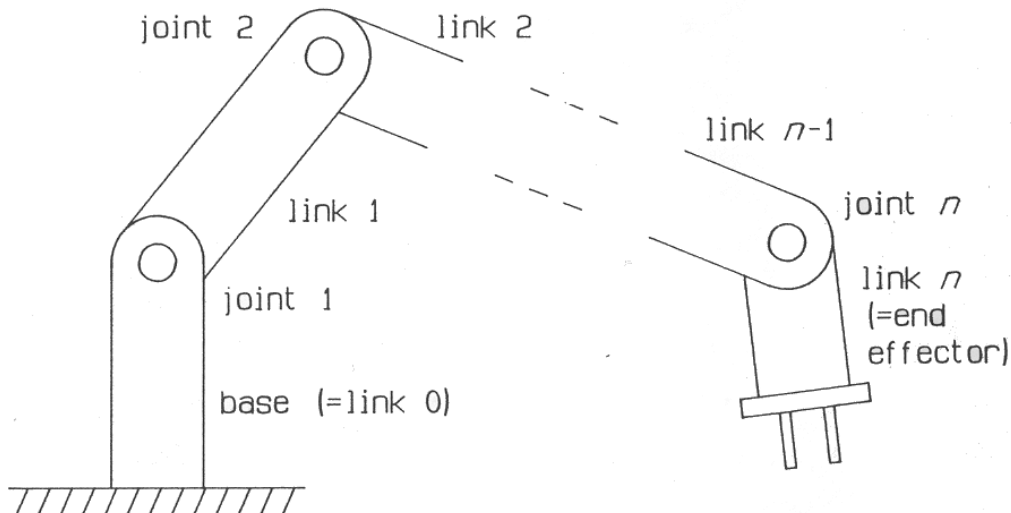
Newton-Euler Algorithm

1. Find the velocities and accelerations of the links

$$\mathbf{v}_i = \mathbf{v}_{i-1} + \Phi_i \dot{q}_i \quad (\mathbf{v}_0 = \mathbf{0})$$

$$\mathbf{a}_i = \mathbf{a}_{i-1} + \Phi_i \ddot{q}_i + \dot{\Phi}_i \dot{q}_i \quad (\mathbf{a}_0 = \mathbf{0})$$

where Φ_i is the axis of rotation of joint i ,
 q , $\dot{q}$ and $\ddot{q}$ are the joint angle, rotation speed
 and acceleration
 and
 $\mathbf{v}_i$ and $\mathbf{a}_i$ are the velocity and acceleration of
 the link in the world

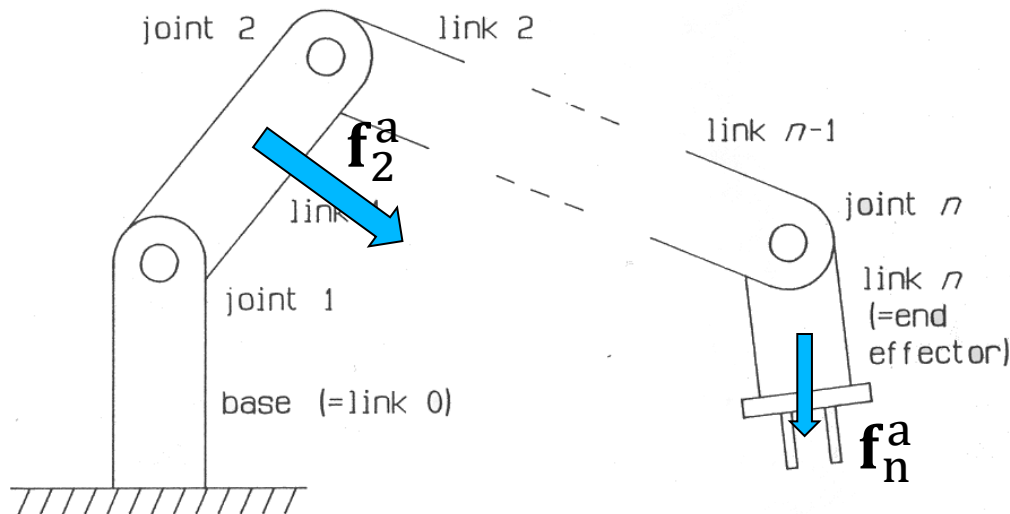


Newton-Euler Algorithm

2. Calculate the equation of motion for each link

$$\mathbf{f}_i^a = \mathbf{I}_i \mathbf{a}_i + \mathbf{v}_i \times \mathbf{I}_i \mathbf{v}_i$$

Computing $\mathbf{f}_i^a$, the total amount of force needed to do the motion



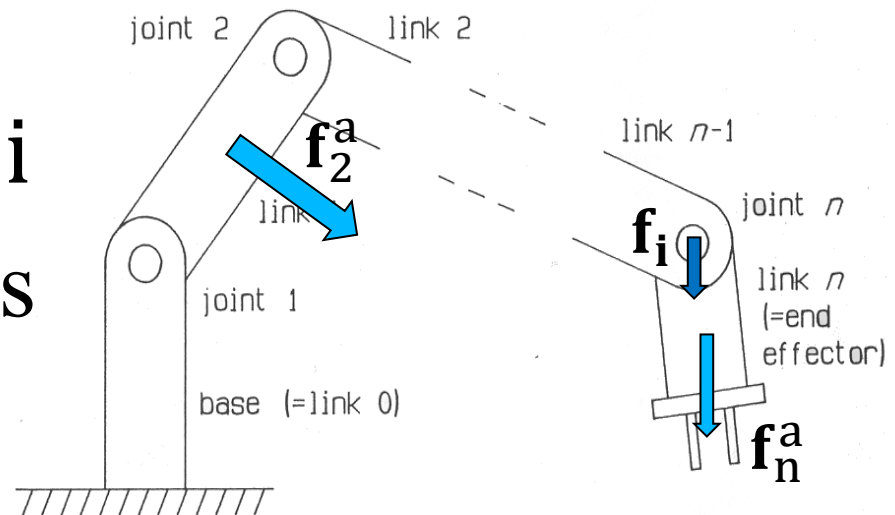
Newton-Euler Algorithm

3. Calculate the joint forces (from leaves towards the root)

$$\mathbf{f}_i^a = \mathbf{f}_i - \mathbf{f}_{i+1} + \mathbf{f}_i^c$$

$$\mathbf{f}_i = \mathbf{f}_i^a - \mathbf{f}_i^c + \mathbf{f}_{i+1}$$

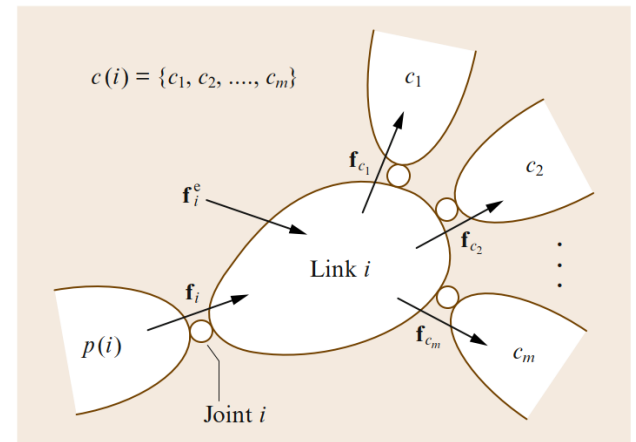
- $\mathbf{f}_i$: force at joint i
- $\mathbf{f}_i^c$: external forces



Newton-Euler Algorithm

3. Calculate the joint forces (from leaves towards the root)

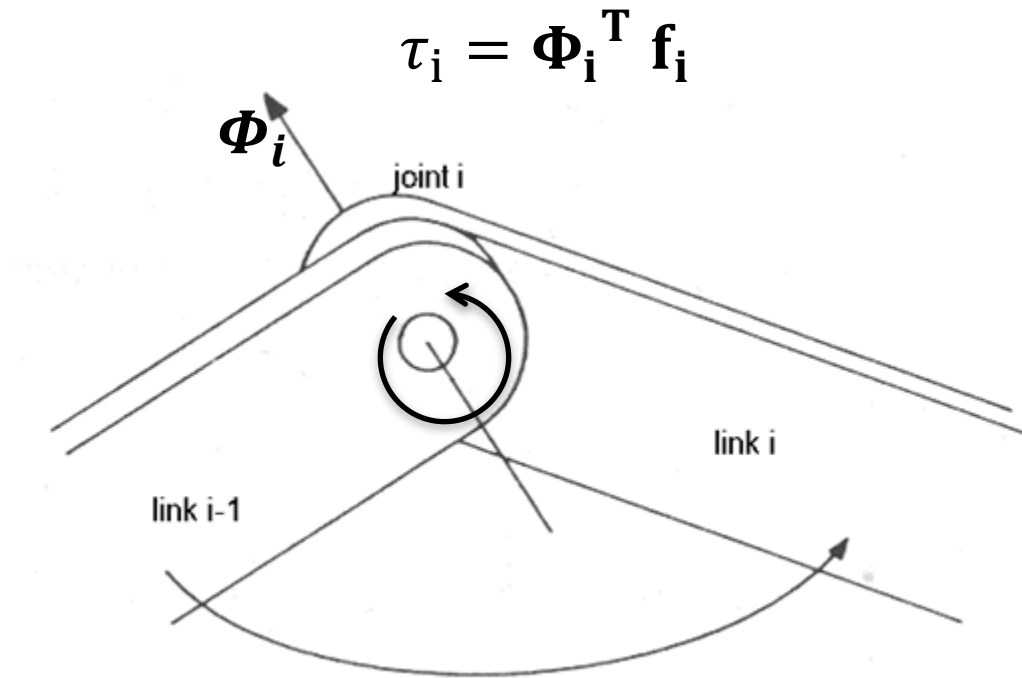
$$\mathbf{f}_i = \mathbf{f}_i^a - \mathbf{f}_i^c + \sum_{j \in c(i)} \mathbf{f}_j$$



- $\mathbf{f}_i$: force at joint i
- $c(i)$: the child joints of link i
- $\mathbf{f}_i^c$: external forces like gravity

Newton-Euler Algorithm

- Calculate the active forces



Inverse Dynamics: Summary

- Inverse Dynamics
 - The calculation of forces given a set of accelerations
 - Newton-Euler : $O(N)$
- Useful for motion analysis, motion synthesis by optimization (covered later)

Forward Dynamics

- The calculation of accelerations given a set of forces
- Needed for simulating the motion in the virtual environment
- Two approaches described here
 - Recursive Newton-Euler Algorithm
 - A method based on inverse dynamics
 - Easy to understand but slow: $O(N^3)$
 - Articulated Body Inertia
 - Faster to compute: $O(N)$
 - Standard approach in robotics

Recursive Newton-Euler Forward Dynamics

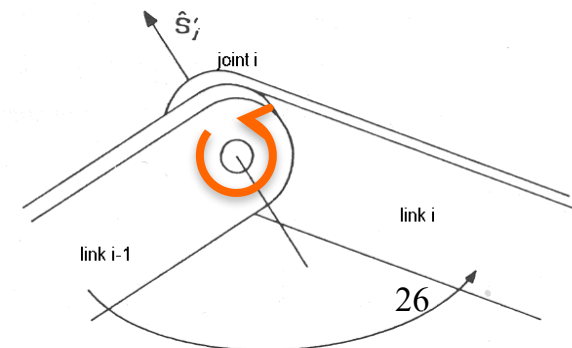
$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

- $\boldsymbol{\tau}$ is the vector of the forces on the links
- $\mathbf{H}$ is the joint-space inertia matrix ($n \times n$)
- $\mathbf{C}$ is the Coriolis term
- $\mathbf{q}, \dot{\mathbf{q}}$ and $\ddot{\mathbf{q}}$ are the coordinates, velocities and accelerations
- To compute $\ddot{\mathbf{q}}$ from $\boldsymbol{\tau}$, we need to know $\mathbf{H}$, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$, $\boldsymbol{\tau}_g$
- Then we can compute $\ddot{\mathbf{q}}$ by solving
$$\mathbf{H}\ddot{\mathbf{q}} = \left(\boldsymbol{\tau} - \boldsymbol{\tau}_g - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \right)$$

Recursive Newton-Euler Forward Dynamics

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

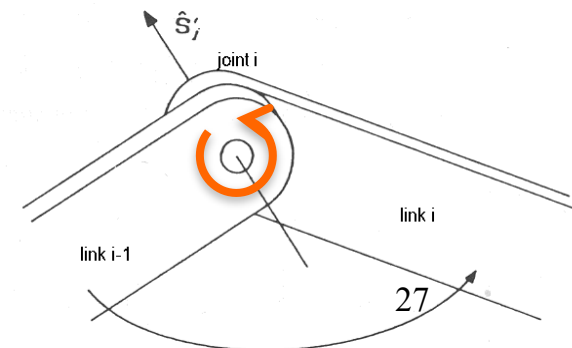
- Algorithm (based on Newton-Euler ID)
 - Calculate $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$
 - Calculate $\mathbf{H}$
 - Solve for $\ddot{\mathbf{q}}$



Recursive Newton-Euler Forward Dynamics

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

- Let's assume we have a module $ID(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$
- This is based on Newton-Euler inverse dynamics
- We can give any $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$ and compute the torque



Recursive Newton-Euler Forward Dynamics

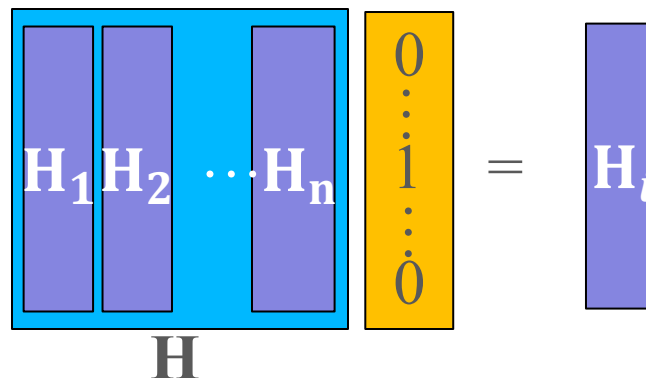
$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

- Compute $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$
 - Setting the acceleration to zero, we get $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$
 - $\text{ID}(\mathbf{q}, \dot{\mathbf{q}}, 0) = \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$
 - We can use the inverse dynamics solver (Newton-Euler) to solve for the forces given the position, velocity and an acceleration of zero

Recursive Newton-Euler Forward Dynamics

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

- Calculate $\mathbf{H}$:
- Again, we use the inverse dynamics solver
- This time we can set $\ddot{\mathbf{q}}$ to $(0, \dots, \underset{\uparrow i}{1}, \dots, 0)$ to compute the i -th column of $\mathbf{H}$
- By subtracting $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$ from the computed $\boldsymbol{\tau}$, we get the i -th column of $\mathbf{H}$



Recursive Newton-Euler Forward Dynamics

- How to construct $\mathbf{H}$? $\mathbf{H}(\mathbf{q})\ddot{\mathbf{q}}$
 - Denote $\mathbf{H} = [\mathbf{H}_1 \mathbf{H}_2 \cdots \mathbf{H}_n]$, $\mathbf{H}_i$: i-th column of $\mathbf{H}$
 - Then $\mathbf{H}_i = ID(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{e}_i) - (\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g)$ (ID :inverse dynamics)
where $\mathbf{e}_i$ is the i-th basis vector (all zero except i-th element 1)
- Algorithm to construct equations of motion
 - $\mathbf{c} = ID(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{0})$
 - for $i=1$ to n
 - $\mathbf{M}_i = ID(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{e}_i) - \mathbf{c}$
 - $\mathbf{H} = [\mathbf{H}_1, \mathbf{H}_2 \dots \mathbf{H}_n]$

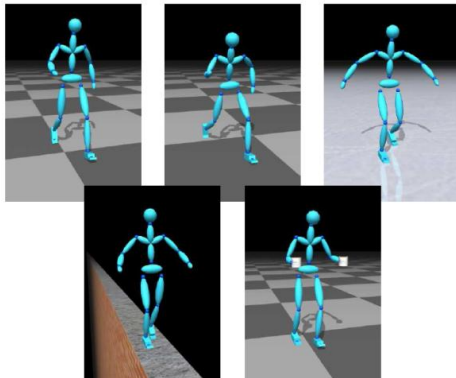

 ID called (N+1) times $\rightarrow O(N^2)$
 Solving for $\mathbf{H}\ddot{\mathbf{q}} = \mathbf{b}$ $O(N^3)$

Articulated-Body Algorithm

- Until now, we've seen a forward dynamics procedure
 - Step 1: Constructing equations of motion $\rightarrow O(N^2)$
 - Step 2: Computing joint acceleration $\rightarrow O(N^3)$
- In fact, there's a much faster algorithm
- Articulated-Body Algorithm
 - Developed by Featherstone in 1983
 - It only takes $O(N)$ time for open-loop system!
 - It is a recursive algorithm like Recursive Newton-Euler inverse dynamics
 - It doesn't construct equations of motion explicitly.
 - If interested, read Ch. 2 of *Handbook of Robotics*, Springer
 - End of slides show the idea of the algorithm

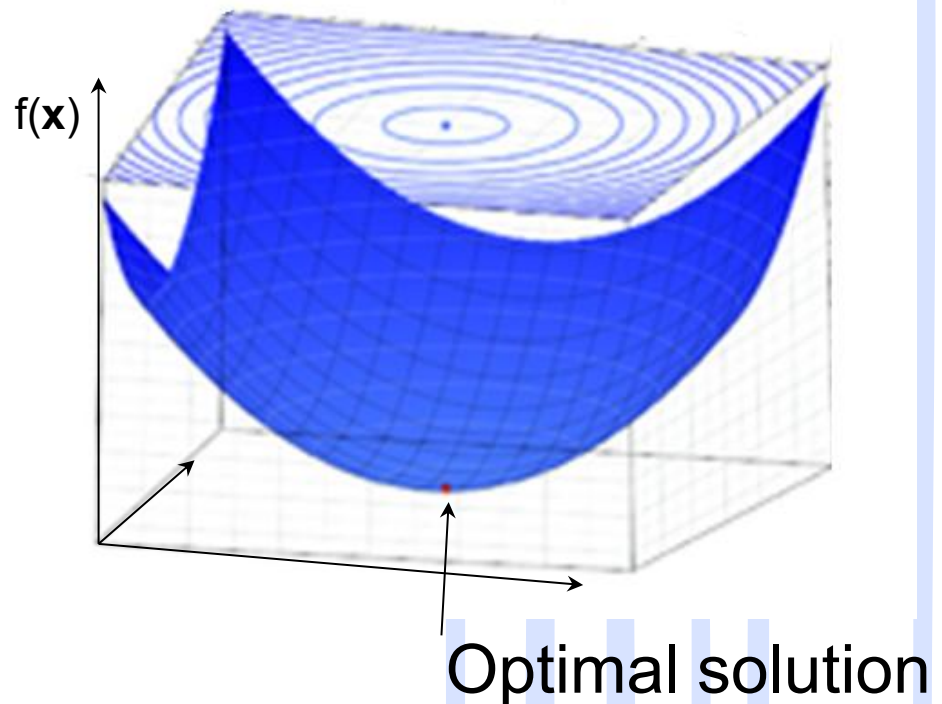
Using Inverse Dynamics to Optimize Motion

- We want the characters to follow the laws of physics
 - Humans move efficiently
- We want characters to not spend too much energy
- We also want them to satisfy constraints
 - passing some specific postures, feet on the ground etc.



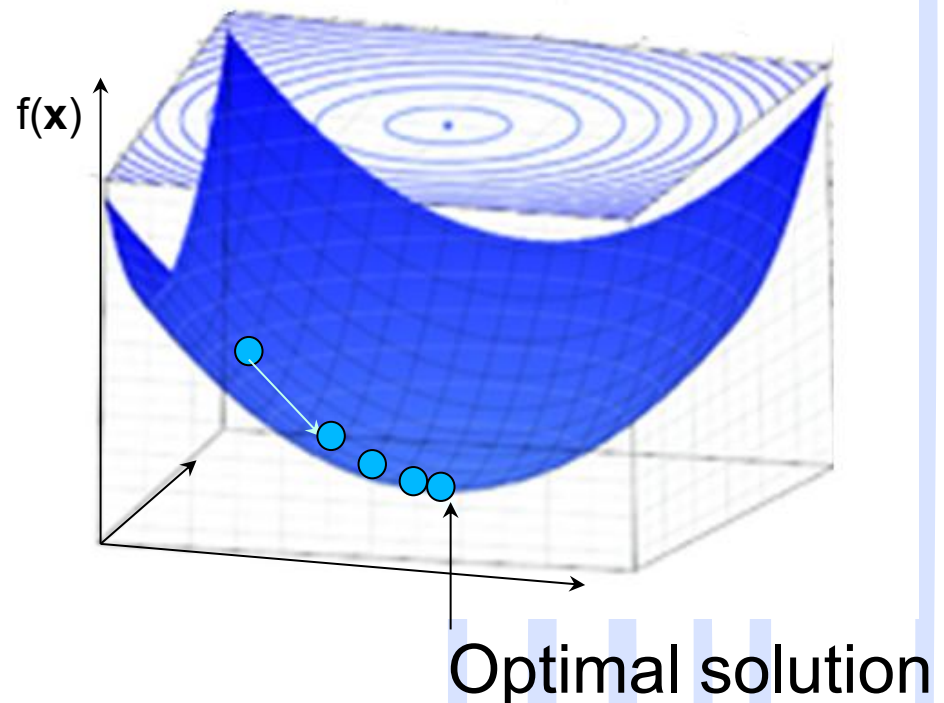
What is Optimization?

- Given an objective function $f(\mathbf{x})$, find the $\mathbf{x}$ that minimizes $f(\mathbf{x})$



What is Optimization?

- Solution is found by iteratively computing the gradient at the current point, and moving in that direction



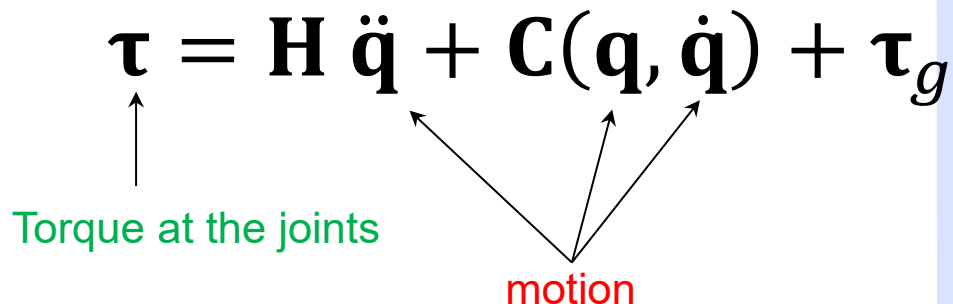
What are the parameters?

- Given some function $f(\mathbf{x})$, find the $\mathbf{x}$ that minimizes $f(\mathbf{x})$
 - Subject to constraints $g(\mathbf{x}) = \mathbf{0}$
- Forward dynamics : Torque at the joints, or parameters of PD control (target postures, gains)
- Inverse dynamics : Trajectory of joint angles (or keyframes)

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

Torque at the joints

motion



parameters





$\mathbf{q}_1 \mathbf{q}_2 \dots \dots \dots$

$\mathbf{q}_n$

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

Difficulty 1: High Dimensionality

- The body has many degrees of freedom
- There are many frames in one motion :
- Even we simplify the control (reducing the number of keyframes), the problem can still be very high dimensional

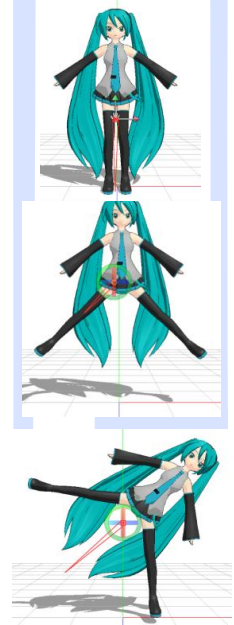
$$\mathbf{q}_1 = (q_1, q_2, q_3, q_4, q_5, q_6, q_7, \dots, q_m)$$

$$\mathbf{q}_2 = (q_1, q_2, q_3, q_4, q_5, q_6, q_7, \dots, q_m)$$

$\vdots$

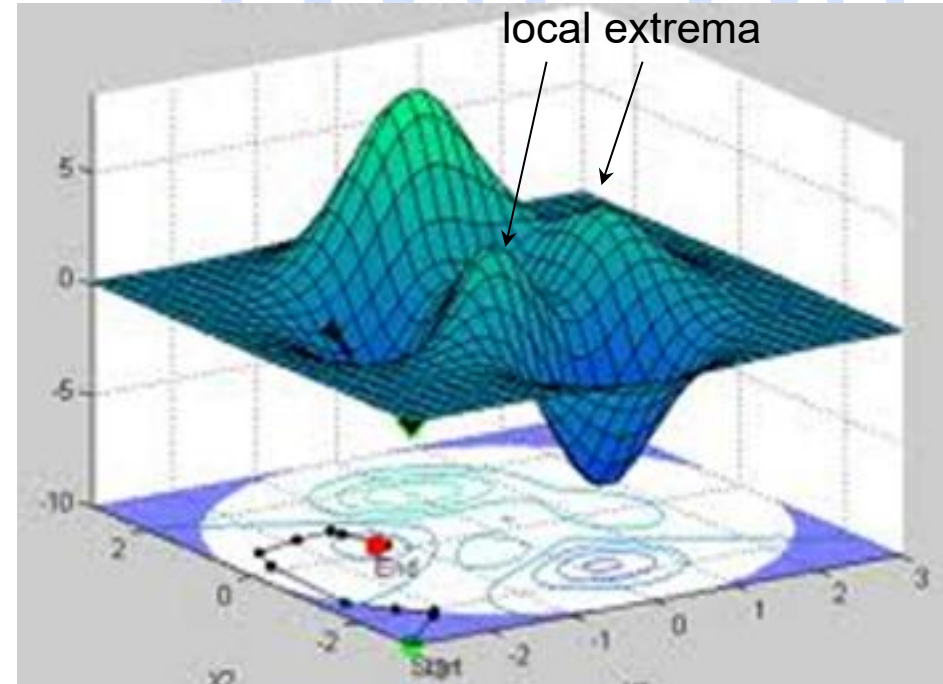
$$\mathbf{q}_n = (q_1, q_2, q_3, q_4, q_5, q_6, q_7, \dots, q_m)$$

- $m \times n$ parameters



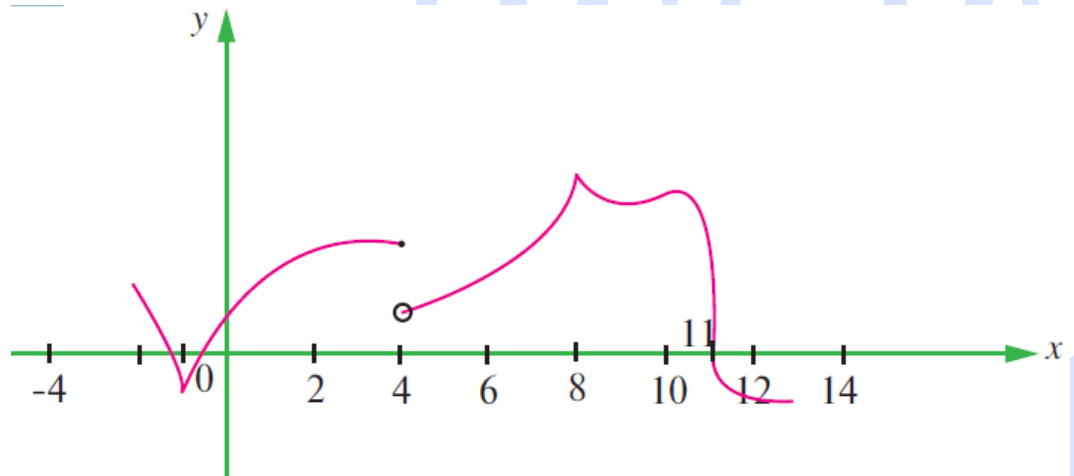
Difficulty 2: Local Extrema

- Functions with many parameters can have lots of local extrema
- An ordinary optimization method can get trapped in local extrema



3: Non-differentiability (forward simulation)

- The motion is nondifferentiable with respect to the parameters, due to the balance/collisions – collisions can suddenly happen and that makes the motion discontinuous with respect to the parameters



What are the parameters?

- Given some function $f(\mathbf{x})$, find the $\mathbf{x}$ that minimizes $f(\mathbf{x})$
 - Subject to constraints $g(\mathbf{x}) = \mathbf{0}$
- Forward dynamics : Torque at the joints, or parameters of PD control (keyframes, coefficients)
- Inverse dynamics : Trajectory of joint angles (or keyframes)

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

Torque at the joints

motion

parameters

What are the parameters?

$$\dot{\mathbf{q}}_i = \frac{\mathbf{q}_{i+1} - \mathbf{q}_{i-1}}{\Delta t}$$

$$\ddot{\mathbf{q}}_i = \frac{\dot{\mathbf{q}}_{i+1} - \dot{\mathbf{q}}_{i-1}}{\Delta t}$$

$$\boldsymbol{\tau}_i = \mathbf{H}_i \ddot{\mathbf{q}}_i + \mathbf{C}(\mathbf{q}_i, \dot{\mathbf{q}}_i) + \boldsymbol{\tau}_i^g$$

Problem to solve

$$\min_q f = \sum_i^n \|\boldsymbol{\tau}_i\|^2$$

subject to

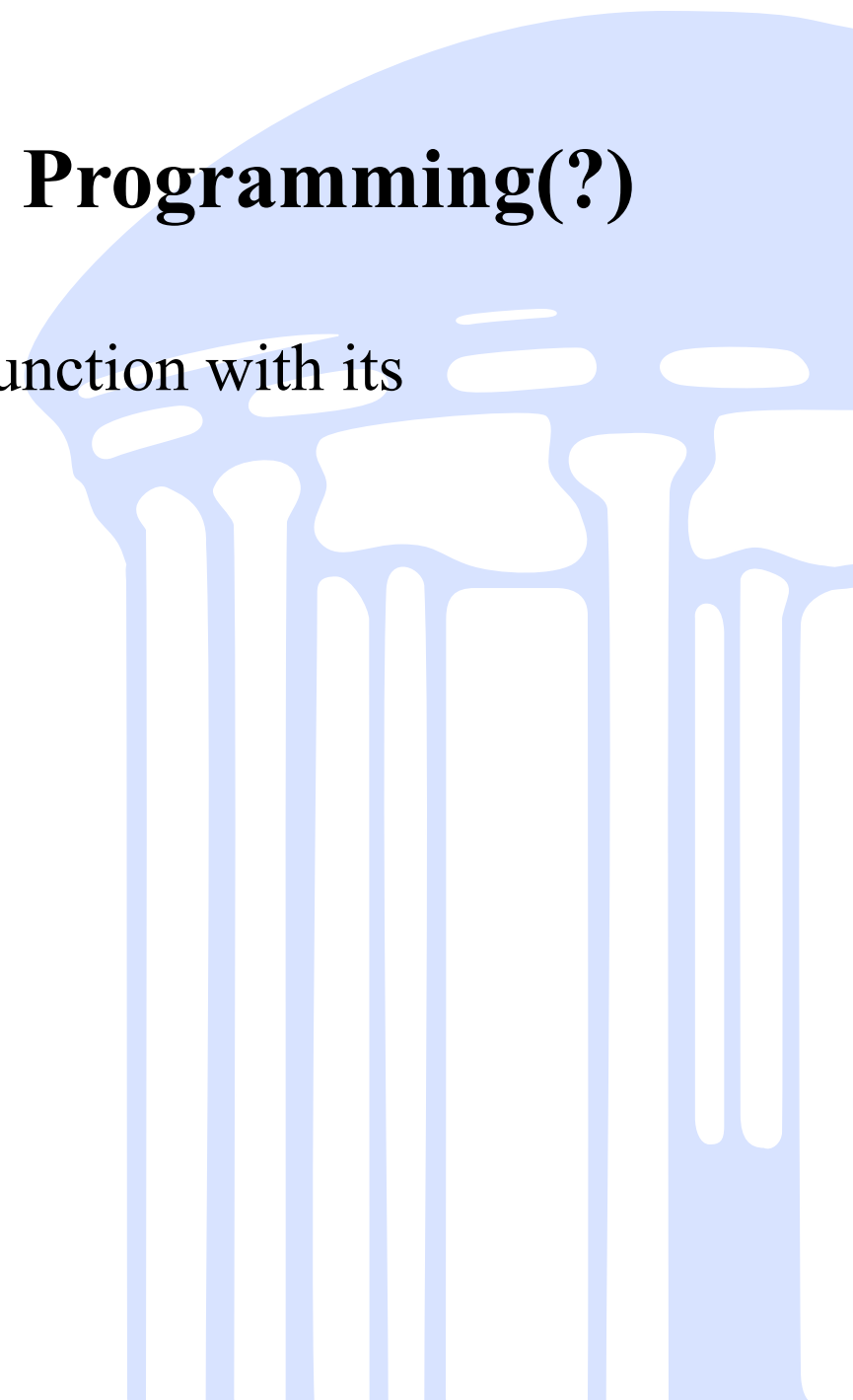
$$\mathbf{q}_0 = \langle \text{first keypose} \rangle$$

$$\mathbf{q}_n = \langle \text{last keypose} \rangle$$

$$\boldsymbol{\tau}_i = \mathbf{H}_i \ddot{\mathbf{q}}_i + \mathbf{C}(\mathbf{q}_i, \dot{\mathbf{q}}_i) + \boldsymbol{\tau}_i^g \quad (i = 0, \dots, n)$$

Sequential Quadratic Programming(?)

Approximate the objective function with its second derivative:



Sequential Quadratic Programming(?)

$$\min_q f = \sum_i^n \|\boldsymbol{\tau}_i\|^2$$

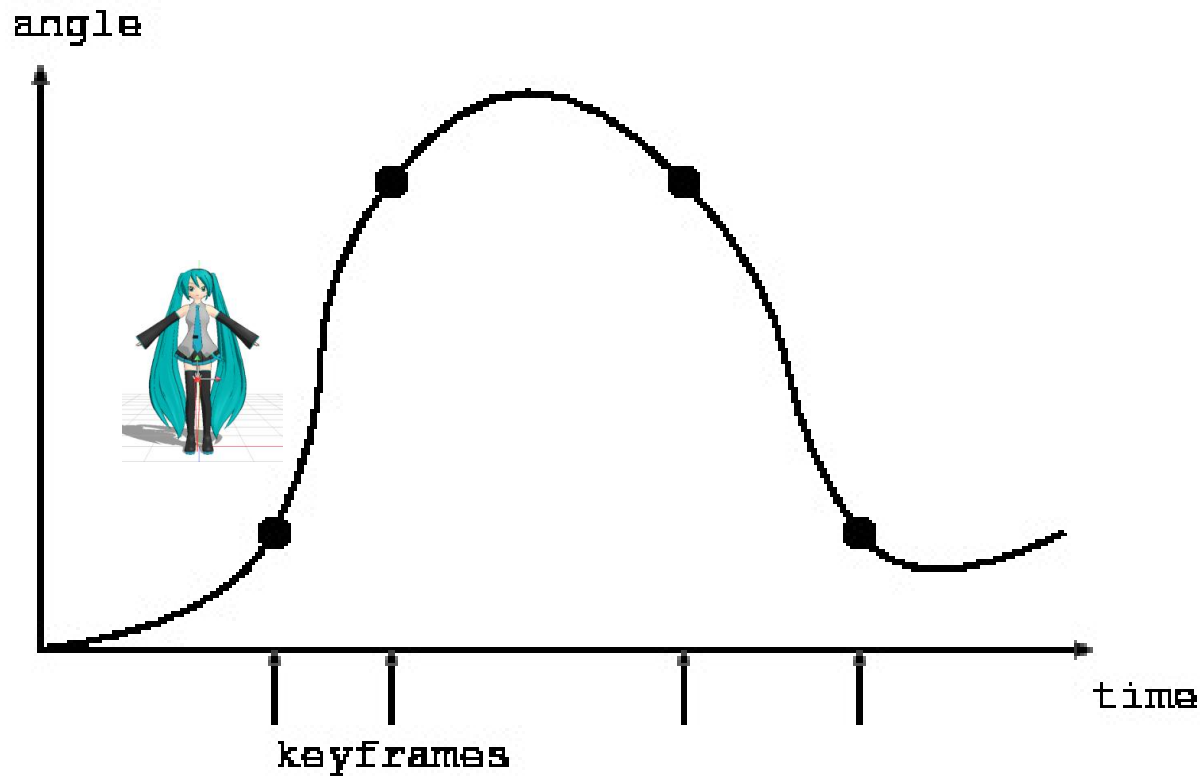
subject to

$\mathbf{q}_0 = \langle \text{first keypose} \rangle$

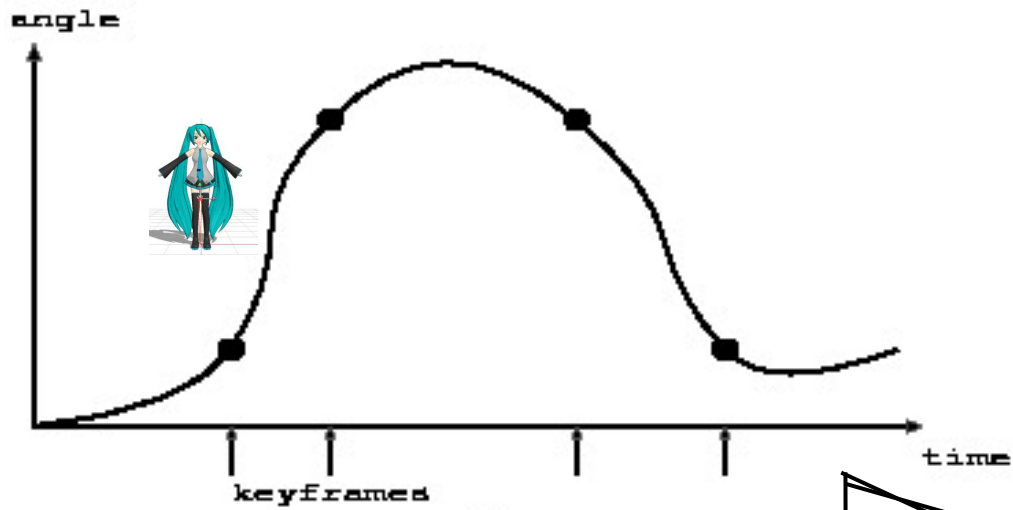
$\mathbf{q}_n = \langle \text{last keypose} \rangle$

$$\boldsymbol{\tau}_i = \mathbf{H}_i \ddot{\mathbf{q}}_i + \mathbf{C}(\mathbf{q}_i, \dot{\mathbf{q}}_i) + \boldsymbol{\tau}_i^g \quad (i = 0, \dots, n)$$

Recap: Kinematic motion representation

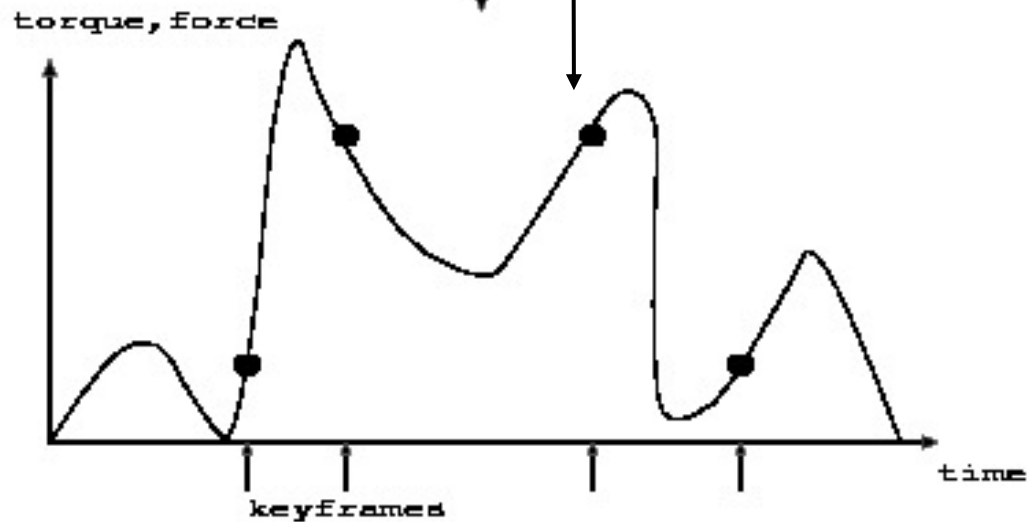


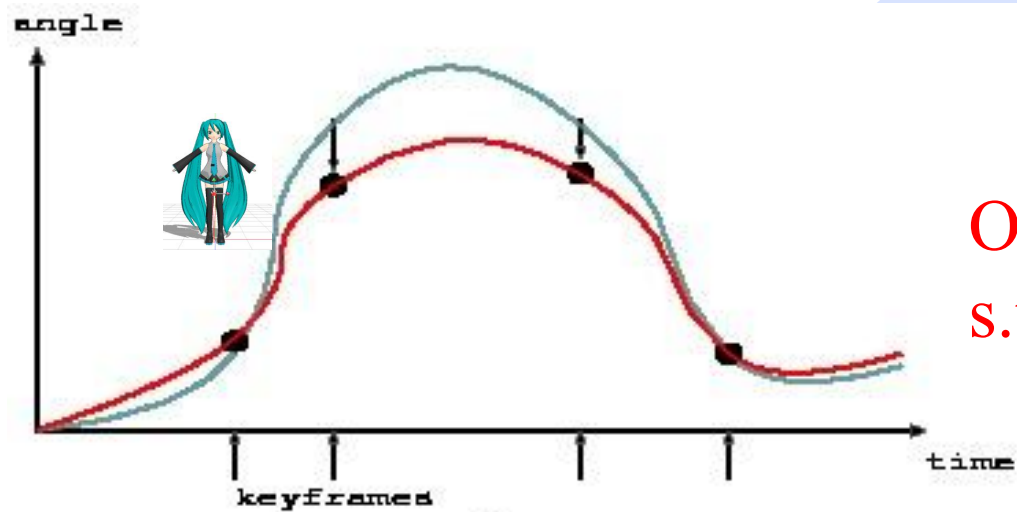
The user gives the keyframes and the motion is computed by interpolation (linear, b-splines, etc)



**Inverse
Dynamics**

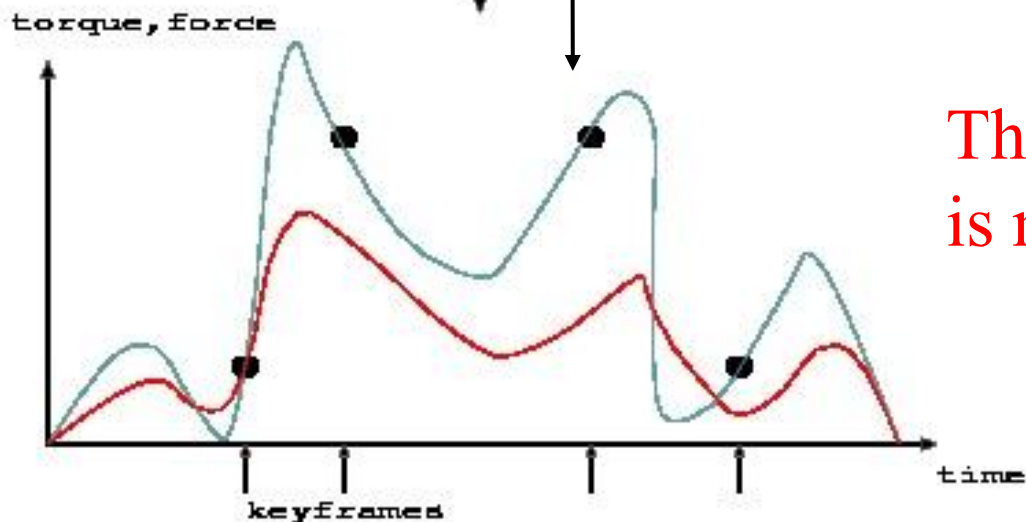
$$\tau = H \ddot{q} + C(q, \dot{q}) + \tau_g$$





Optimize the keyframes
s.t.

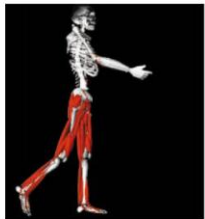
Inverse
Dynamics



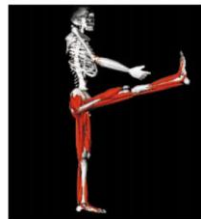
The required force
is minimal

Optimization in Inverse Dynamics

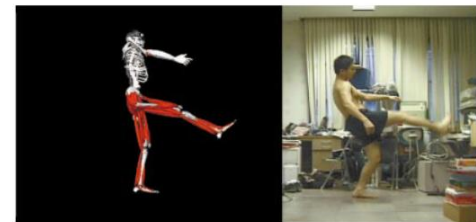
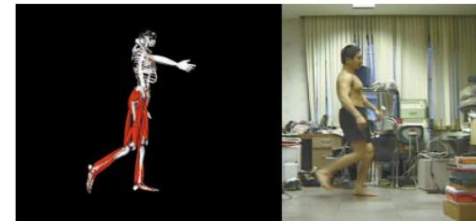
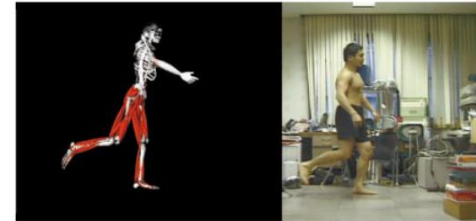
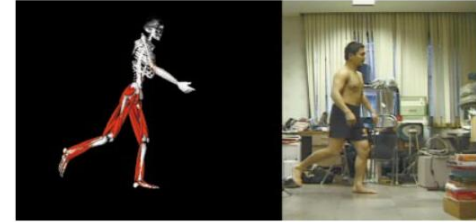
- Given the first and second pose to specify the initial motion, optimize the motion such that
- balance is preserved, and
- muscles can afford the motion



a



b





Optimization in Inverse Dynamics

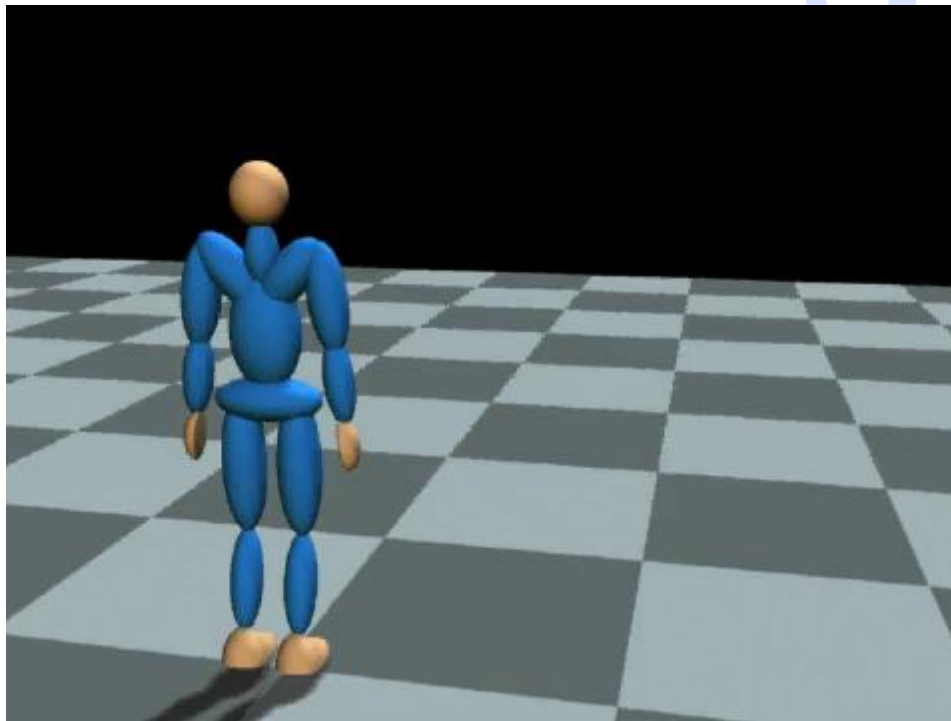
- Objective Function: $f(\mathbf{x}) = \|\tau\|^2 + \|\tau_{feet}\|^2$
- τ : Torque produced at the joints
- $\mathbf{x}$: The parameters of the motion – here the control points of the uniform cubic B-Splines
- τ_{feet} : torque needed to be added to the supporting foot to keep the balance

Solved by nonlinear optimization

Optimizing Motion Considering Physics (Liu and Popovic 2002)

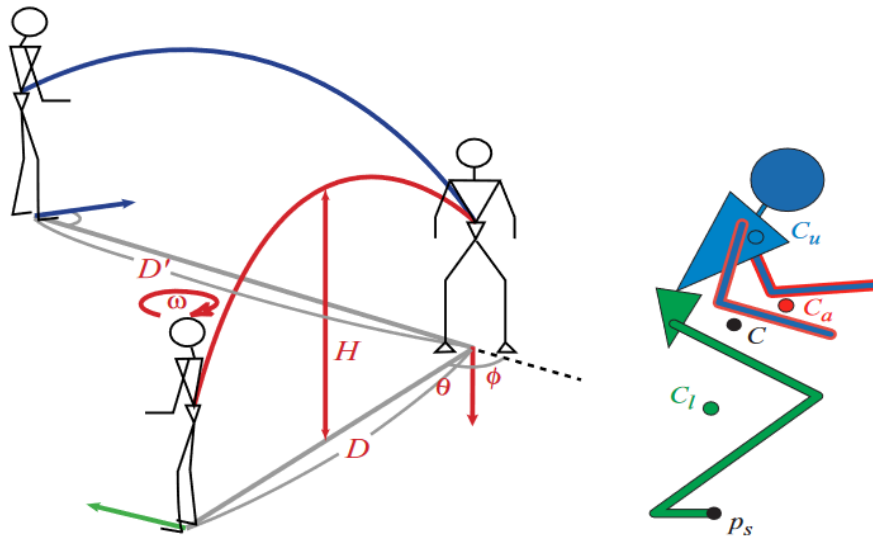
- Compute the trajectories of the body such that the body follows the user inputs and produce natural motion

Liu and Popovic '02



Optimizing Motion Considering Physics (Liu and Popovic 2002)

- The motion is divided into the unconstrained stage (in the air) and constrained stage (on the ground)



Optimizing Motion Considering Physics (Liu and Popovic 2002)

Constraints

- Momentum constraint (when in the air)
- Feet position on the ground, specified by the user

Objective function

- Minimum mass displacement
- Minimal velocity of DOFs
- Static balance (when on the ground)

Optimizing Motion Considering Physics (Liu and Popovic 2002)

Momentum constraint

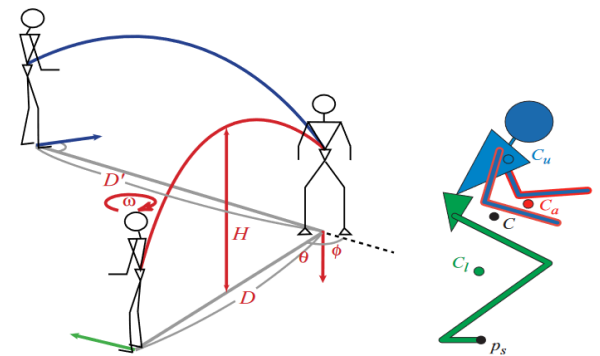
In the air :

- Linear momentum only affected by gravity

$$d\mathbf{P}(\mathbf{q})/dt = m\ddot{\mathbf{C}}(\mathbf{q}) = m\mathbf{g},$$

- Angular momentum : No changes

$$d\mathbf{L}(\mathbf{q})/dt = \mathbf{0}.$$



Optimizing Motion Considering Physics (Liu and Popovic 2002)

Objective Function: Minimum mass displacement

- We prefer to minimize the motion or energy

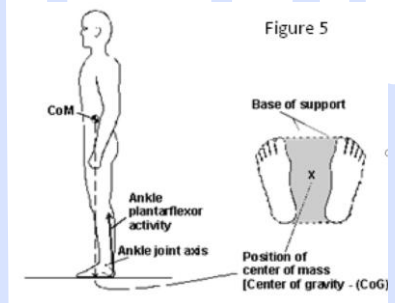
$$E_k = \iiint_i \mu_i (\mathbf{p}_i - \bar{\mathbf{p}}_i)^2 dx dy dz,$$

- Objective Function: Minimum joint angle motion

$$E = \int \|\dot{\mathbf{q}}\|^2 dt$$

- Static balance

Distance to the base of support boundary



Summary

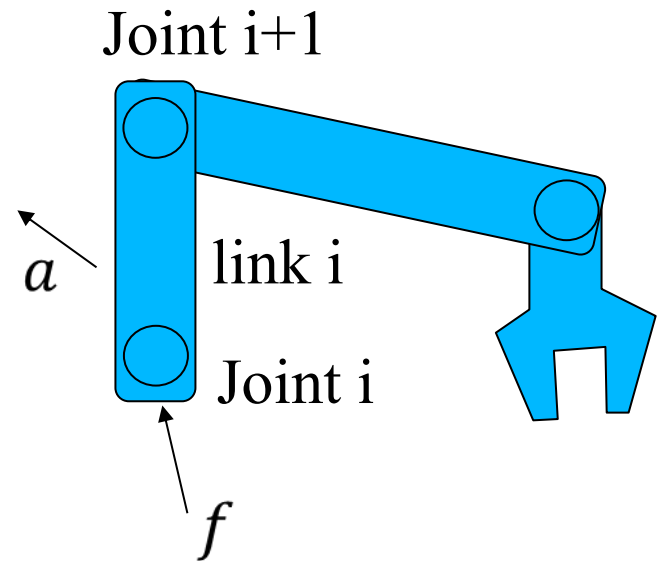
- Inverse dynamics : computing force/torque from motion
- Forward dynamics : computing motion from force/torque
- Inverse dynamics useful for analysis, control
 - Also useful for optimizing the motion
- Forward dynamics is needed for physical simulation
 - We can also optimize the torque to simulation realistic motion
 - That is what is done with deep reinforcement learning

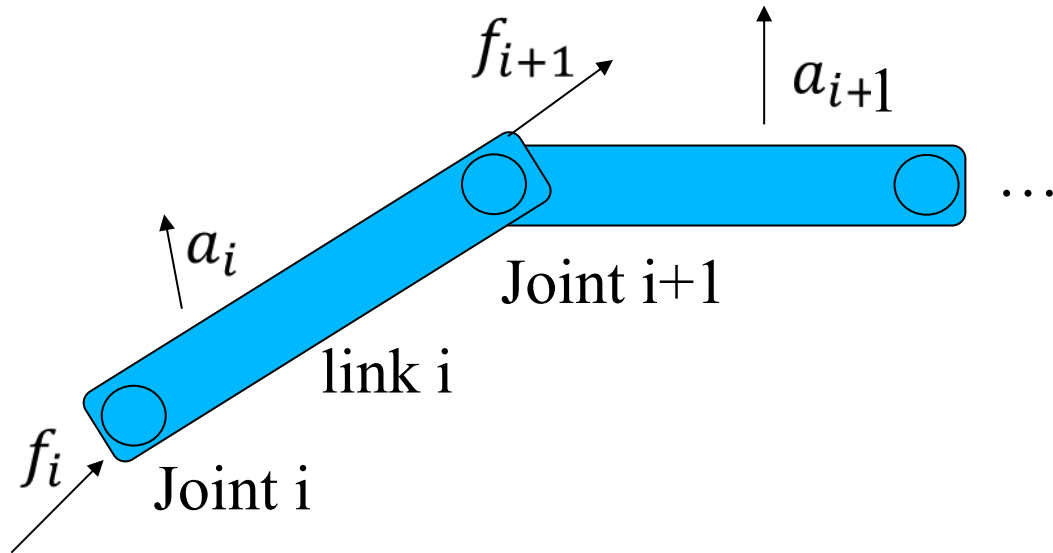
Ref: Articulated Body Inertia

There is a linear relationship between the acceleration of the body and the force applied to it from the joint

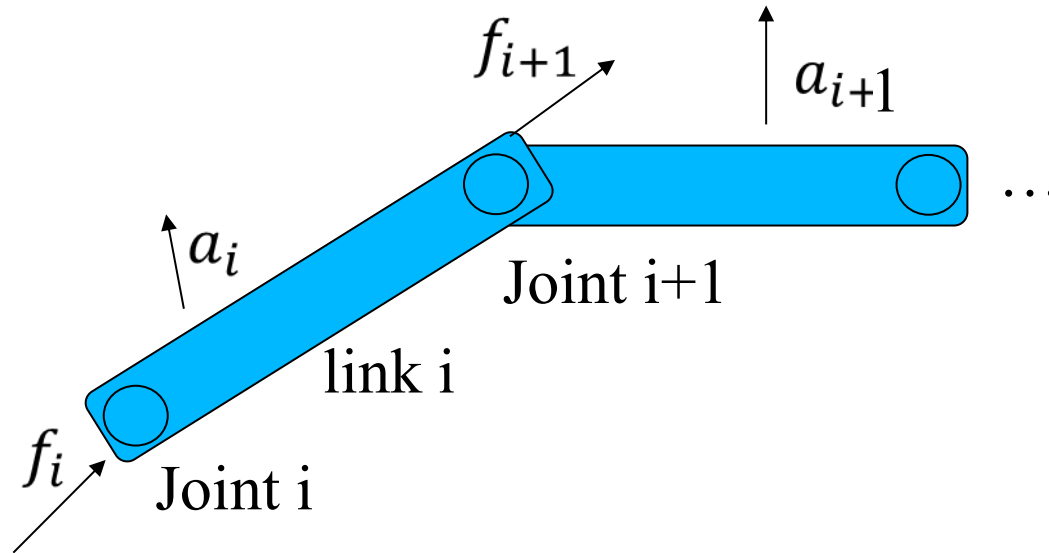
$$\mathbf{f} = \mathbf{I}^A \mathbf{a} + \mathbf{p}$$

where $\mathbf{f}$ is a force applied to a link, $\mathbf{I}^A$ is the articulate body inertia and $\mathbf{p}$ is the bias force, or the force required to keep the link from accelerating





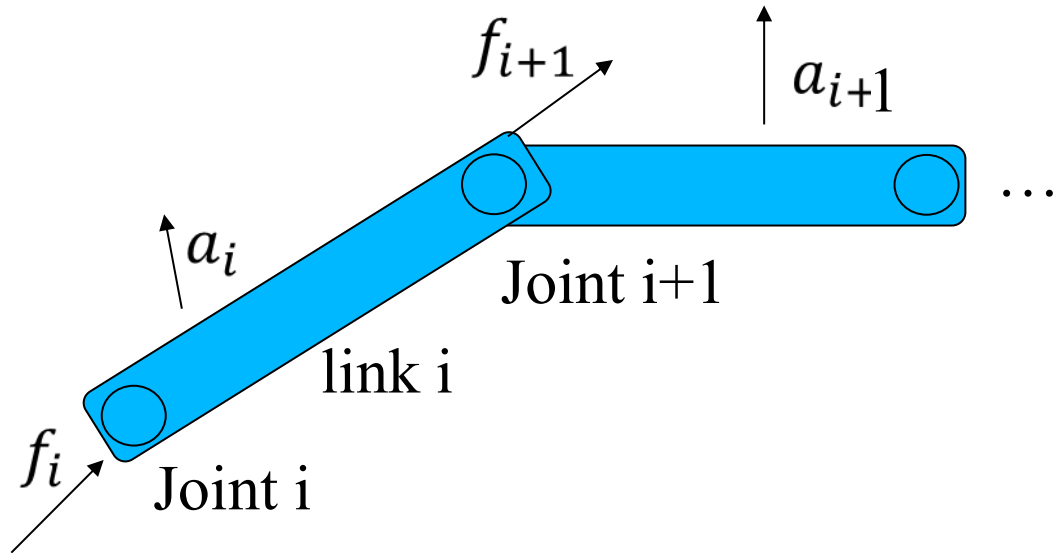
- Let's think about the articulated body inertia of two bodies
- $\mathbf{f}_i = \mathbf{I}_i^A \mathbf{a}_i + \mathbf{P}_i$, $\mathbf{f}_{i+1} = \mathbf{I}_{i+1}^A \mathbf{a}_{i+1} + \mathbf{P}_{i+1}$
- We wish to derive a recursive equation that computes $\mathbf{I}_i^A$, $\mathbf{P}_i$ from $\mathbf{I}_{i+1}^A$, $\mathbf{P}_{i+1}$



- The relation between the acceleration of the first and second link

$$- a_{i+1} = a_i + \Phi_{i+1} \ddot{q}$$

- The relation between the force and torque at link i+1 : $\tau_{i+1} = \Phi_{i+1}^T f_{i+1}$

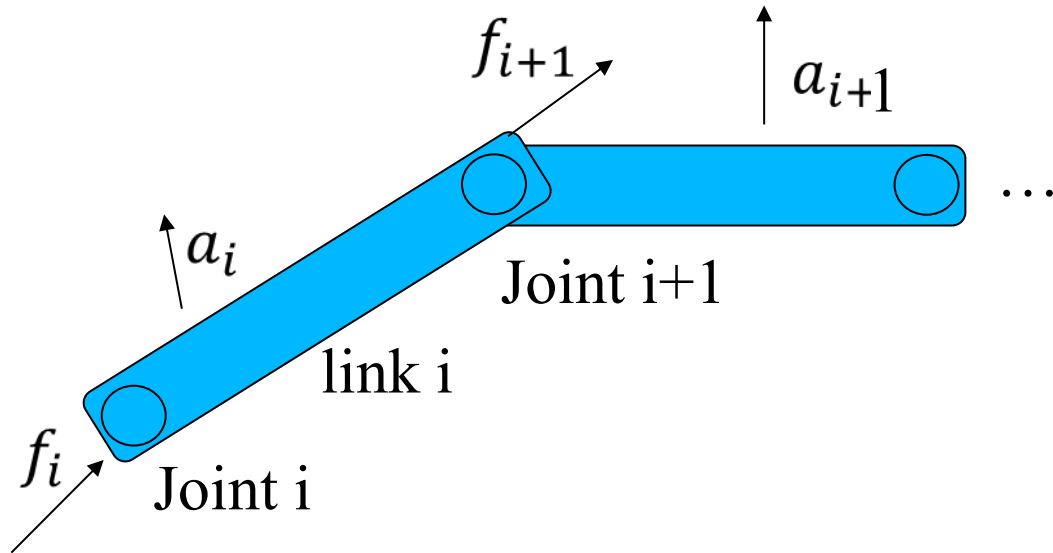


- We try to represent $\ddot{q}$ with τ_{i+1} and $\mathbf{I}_{i+1}^A, \mathbf{P}_{i+1}, \mathbf{a}_i$

We substitute the equation

$$\mathbf{f}_{i+1} = \mathbf{I}_{i+1}^A \mathbf{a}_{i+1} + \mathbf{P}_{i+1}$$

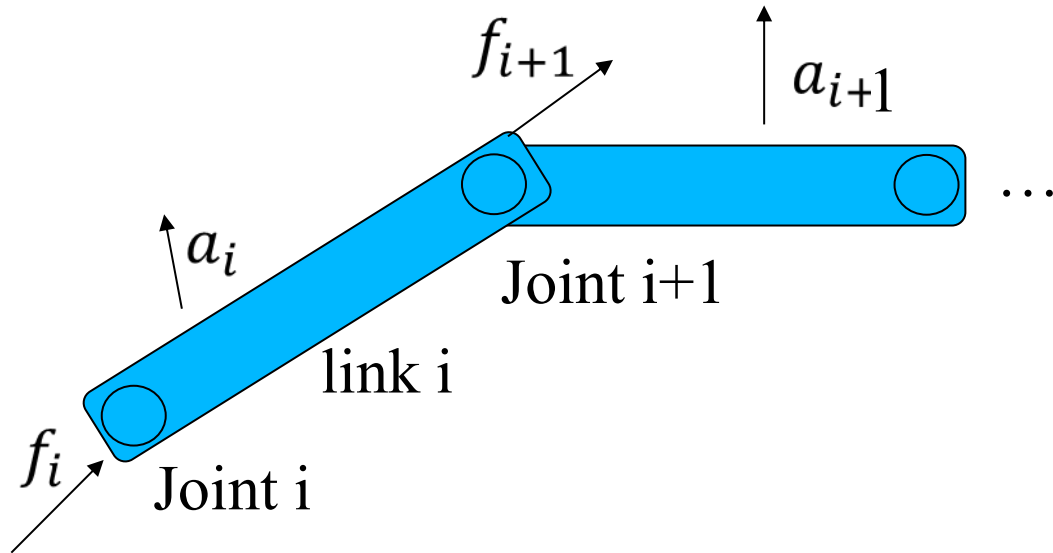
$$\tau_{i+1} = \Phi_{i+1}^T \mathbf{f}_{i+1}$$



$$\tau_{i+1} = \Phi_{i+1}^T (\mathbf{I}_{i+1}^A \mathbf{a}_{i+1} + \mathbf{P}_{i+1})$$

Then substitute $\mathbf{a}_{i+1} = \mathbf{a}_i + \Phi_{i+1} \ddot{\mathbf{q}}$
 into this and solve for $\ddot{\mathbf{q}}$

$$\ddot{\mathbf{q}} = \frac{\tau_{i+1} - \Phi_{i+1}^T \mathbf{I}_{i+1}^A \mathbf{a}_i - \Phi_{i+1}^T \mathbf{P}_{i+1}}{\Phi_{i+1}^T \mathbf{I}_{i+1}^A \Phi_{i+1}}$$

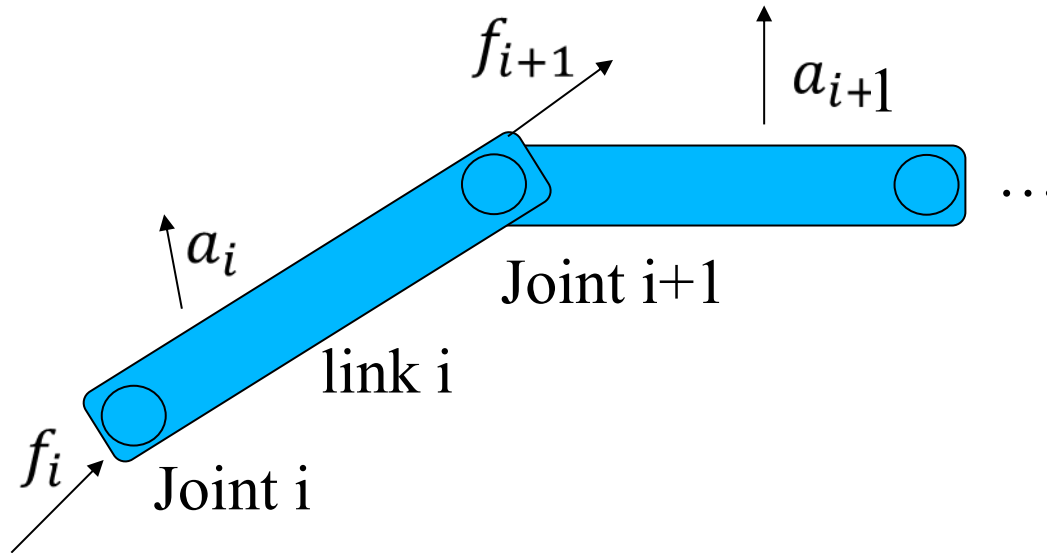


- Now we go back to the first equation

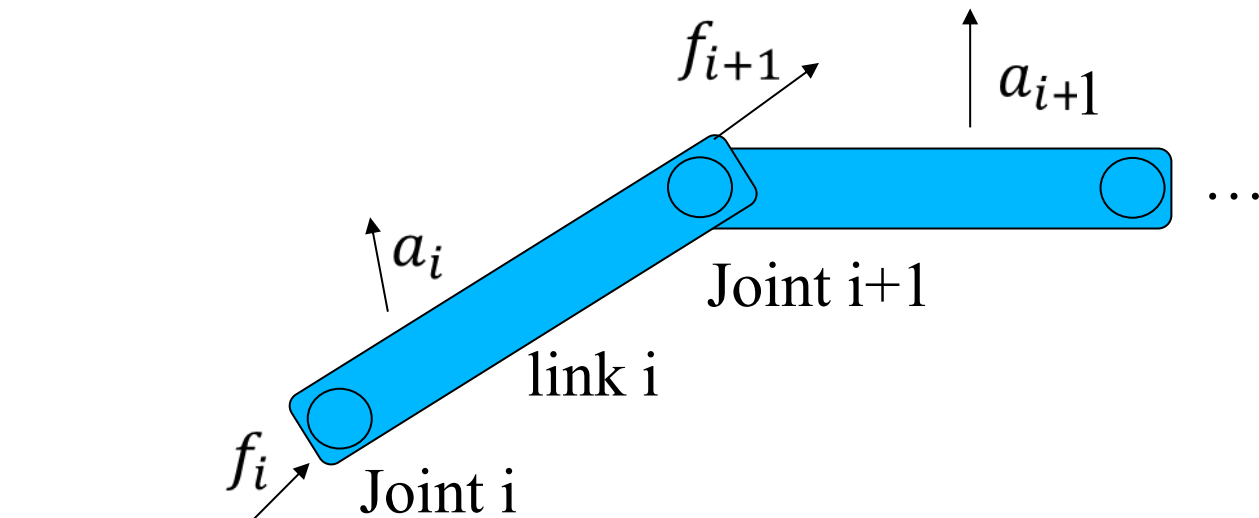
$$\mathbf{f}_i = \mathbf{I}_i^A \mathbf{a}_i + \mathbf{P}_i$$

But we also had $\mathbf{I}_i \mathbf{a}_i = \mathbf{f}_i - \mathbf{f}_{i+1}$

$$\text{So } \mathbf{f}_i = \mathbf{I}_i^A \mathbf{a}_i + \mathbf{P}_i = \mathbf{I}_i \mathbf{a}_i + \mathbf{f}_{i+1}$$



- $$\begin{aligned} \mathbf{f}_i &= \mathbf{I}_i^A \mathbf{a}_i + \mathbf{P}_i = \mathbf{I}_i \mathbf{a}_i + \mathbf{f}_{i+1} \\ &= \mathbf{I}_i \mathbf{a}_i + \mathbf{I}_{i+1}^A \mathbf{a}_{i+1} + \mathbf{P}_{i+1} \\ &= \mathbf{I}_i \mathbf{a}_i + \mathbf{I}_{i+1}^A (\mathbf{a}_i + \boldsymbol{\Phi}_{i+1} \ddot{\mathbf{q}}) + \mathbf{P}_{i+1} \\ &= \mathbf{I}_i \mathbf{a}_i + \mathbf{I}_{i+1}^A \left(\mathbf{a}_i + \boldsymbol{\Phi}_{i+1} \frac{\tau_{i+1} - \boldsymbol{\Phi}_{i+1}^T \mathbf{I}_{i+1}^A \mathbf{a}_i - \boldsymbol{\Phi}_{i+1}^T \mathbf{P}_{i+1}}{\boldsymbol{\Phi}_{i+1}^T \mathbf{I}_{i+1}^A \boldsymbol{\Phi}_{i+1}} \right) + \mathbf{P}_{i+1} \end{aligned}$$

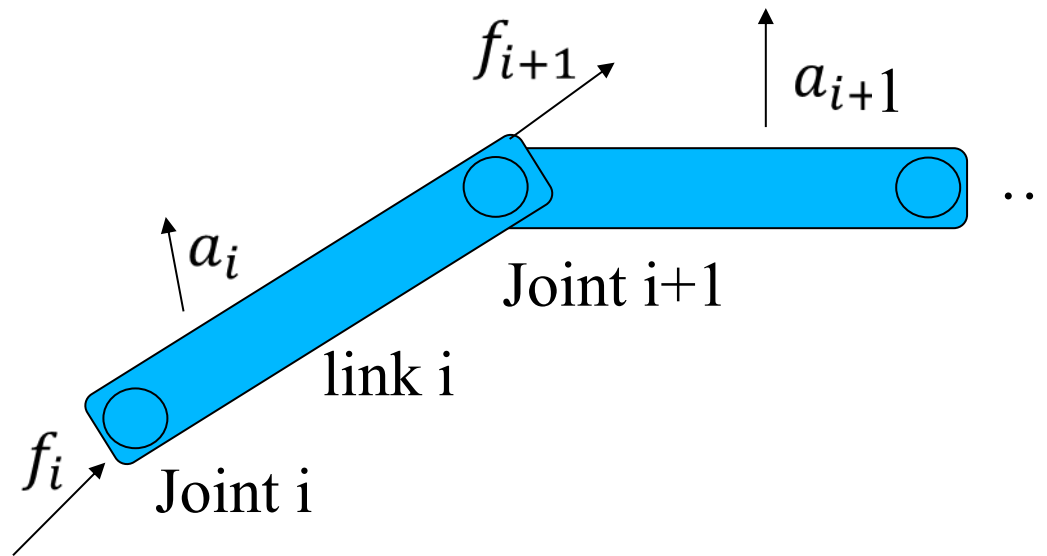


$$\begin{aligned}
 \mathbf{f}_i &= \mathbf{I}_i^A \mathbf{a}_i + \mathbf{P}_i \\
 &\vdots \\
 &= \mathbf{I}_i \mathbf{a}_i + \mathbf{I}_{i+1}^A \left(\mathbf{a}_i + \Phi_{i+1} \frac{\tau_{i+1} - \Phi_{i+1}^T \mathbf{I}_{i+1}^A \mathbf{a}_i - \Phi_{i+1}^T \mathbf{P}_{i+1}}{\Phi_{i+1}^T \mathbf{I}_{i+1}^A \Phi_{i+1}} \right) + \mathbf{P}_{i+1} \\
 &= \left[\mathbf{I}_i + \mathbf{I}_{i+1}^A + \frac{-\Phi_{i+1}^T \mathbf{I}_{i+1}^A}{\Phi_{i+1}^T \mathbf{I}_{i+1}^A \Phi_{i+1}} \right] \mathbf{a}_i + \left[\frac{\tau_{i+1} - \Phi_{i+1}^T \mathbf{P}_{i+1}}{\Phi_{i+1}^T \mathbf{I}_{i+1}^A \Phi_{i+1}} + \mathbf{P}_{i+1} \right]
 \end{aligned}$$

$\uparrow$ $\mathbf{I}_i^A$ $\uparrow$ $\mathbf{P}_i$

We have a recursive form for $\mathbf{I}_i^A, \mathbf{P}_i$!!

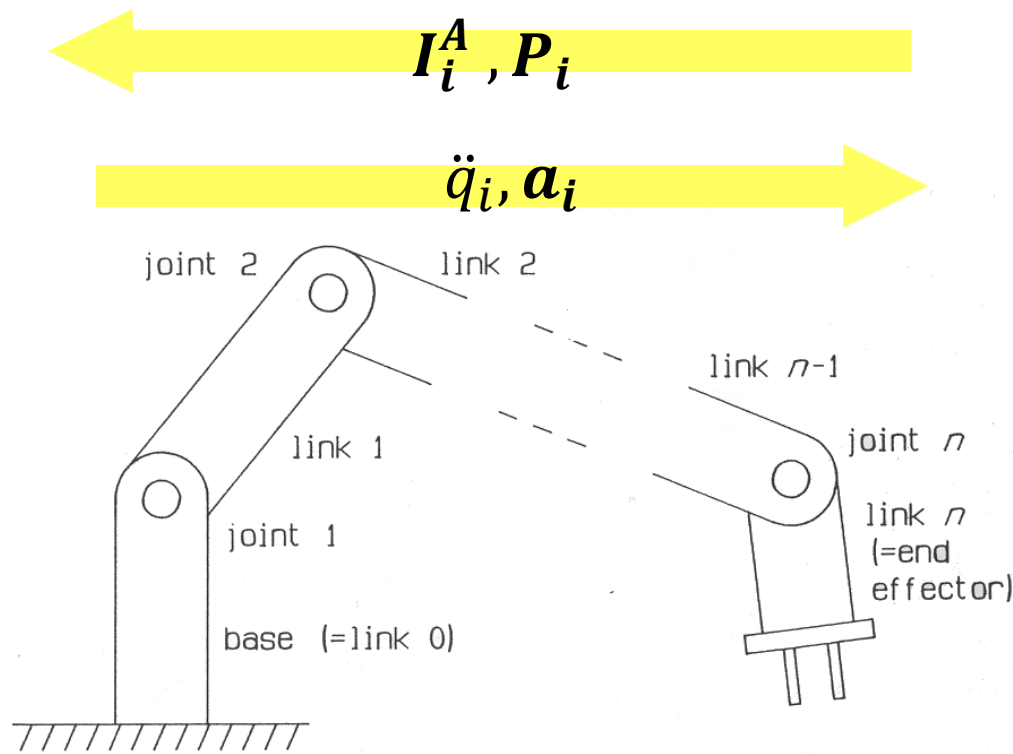
Recursive form for Articulated-Moment Inertia and Bias Force



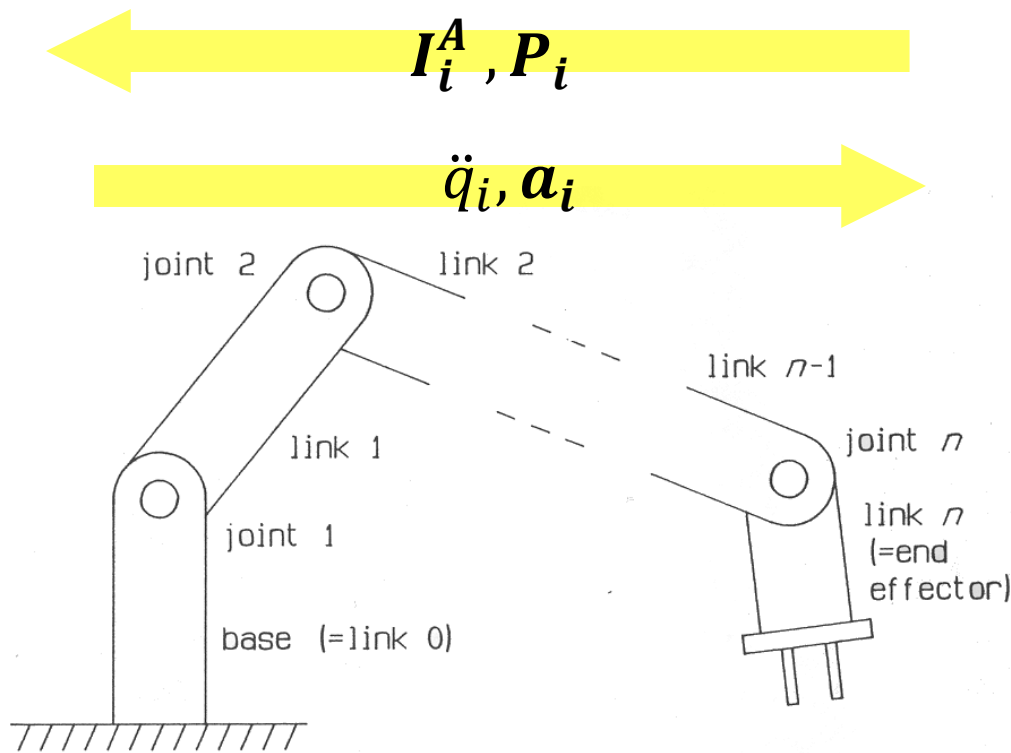
$$\mathbf{I}_i^A = \mathbf{I}_i + \mathbf{I}_{i+1}^A + \frac{-\Phi_{i+1}^T \mathbf{I}_{i+1}^A}{\Phi_{i+1}^T \mathbf{I}_{i+1}^A \Phi_{i+1}} \quad (\mathbf{I}_n^A = \mathbf{I}_n)$$

$$\mathbf{P}_i = \frac{\tau_{i+1} - \Phi_{i+1}^T \mathbf{P}_{i+1}}{\Phi_{i+1}^T \mathbf{I}_{i+1}^A \Phi_{i+1}} + \mathbf{P}_{i+1} \quad (\mathbf{P}_n = \mathbf{0})$$

Start from these



- Compute I_i^A, P_i from leaf to root
- Compute the acceleration $\ddot{q}_i, a_i$ from root to the leaves



Acceleration recursively from root to leaves by

$$\ddot{q}_i = \frac{\tau_i - \Phi_i^T I_i^A a_{i-1} - \Phi_i^T P_i}{\Phi_i^T I_i^A \Phi_i}$$

$$a_i = a_{i-1} + \Phi_i \ddot{q}_i \quad (a_i = 0)$$

Summary of Forward Dynamics

- Let's define a function FD (forward dynamics)

$$\ddot{q} = FD(q, \dot{q}, \tau)$$

We looked at two approaches to realize $\ddot{q} = FD(q, \dot{q}, \tau)$

1. Construct equations of motion and solve it
 - Typically $O(N^3)$ algorithms
 - Composite-rigid-body method is efficient in this kind
2. Articulated-body algorithm
 - $O(N)$ algorithm

References

- R. Featherstone, The Calculation of Robot Dynamics Using Articulated-Body Inertias, IJRR 1983
- Springer Handbook of Robotics Ch. 2 – Can read online from the library
- Roy Featherstone's home page
 - <http://royfeatherstone.org/abstracts.html#RAM-tutorial-1>